Handling Imbalanced Datasets

Lesson Structure

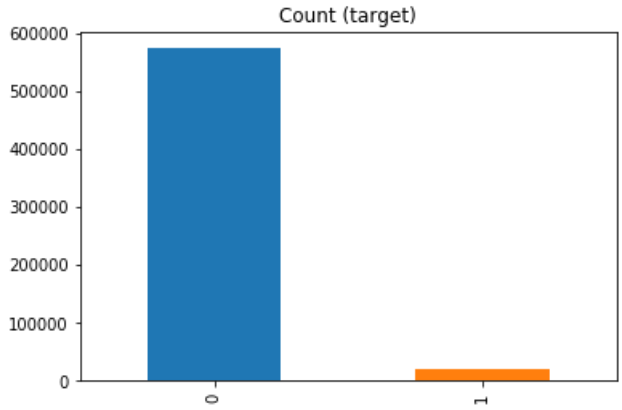
Imbalanced Data
Why It Causes Problems
How to Deal with Imbalanced Data
- **Resampling**
- Model-**level methods**
- **Evaluation Metrics**

▼ Imbalanced Data

An imbalanced dataset is a dataset where one or more labels make up the majority of the dataset, leaving far fewer examples of other labels.

This problem applies to both **classification** and **regression** tasks.

- Classification: binary classification, multiclass classification, multilabel classification.
  - e.g. 95% of labels is in one class.



Credit: Rafael Alencar, https://www.kaggle.com/code/rafjaa/resampling-strategies-for-imbalanced-datasets/notebook

- Regression: examples with outlier values that are either much lower or higher than the median/average of the data.
  - e.g. Predict prices for houses. Houses worth > $10M are rare.

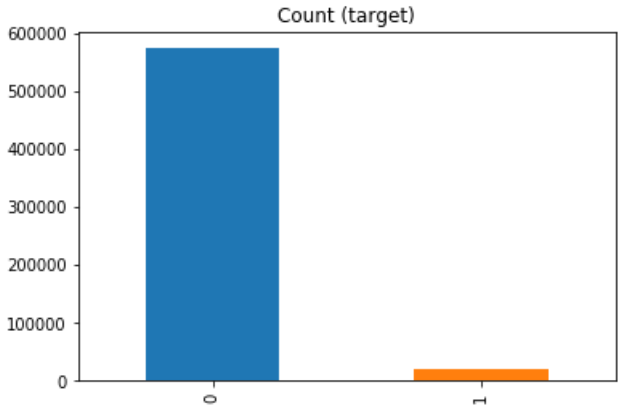
In many scenarios, getting more data for the minority class may be impractical or hard to acquire because the data is **inherently imbalanced**.

- e.g., fraud detection and detection of rare diseases.

▼ Why It Causes Problems

> 🙁 The model cannot learn to predict the minority class well because of class imbalance.

- Model is only able to learn a simple heuristic (e.g. always predict the dominate class) and it gets stuck in a suboptimal solution.
- An accuracy of over 90% can be misleading because the model may not have predictive power on the rare class.
  - e.g. 95% of labels is in one class.



Credit: Rafael Alencar, https://www.kaggle.com/code/rafjaa/resampling-strategies-for-imbalanced-datasets/notebook

- Often, the minority class is more important than the majority class. A wrong prediction on an example of the minority class is more costly than a wrong prediction on an example of the majority class.
    - e.g., Missing a fraudulent transaction is 100x more costly than misclassifying a legitimate example as fraud.

How to Deal with Imbalanced Data

- Data-level: Resampling
- Model-level
- Metric-level

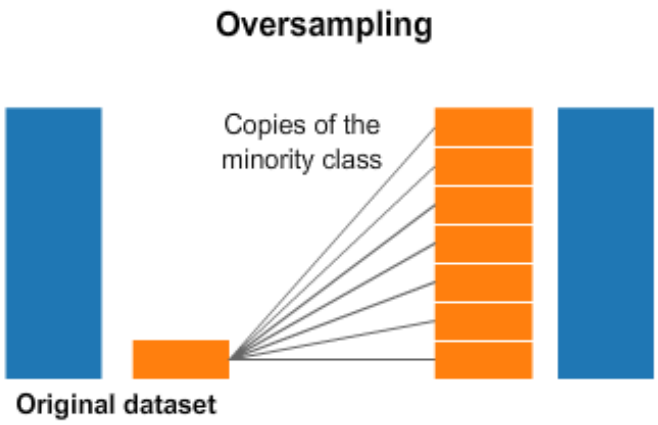
▼ Resampling

Change the distribution of the training data to reduce the level of class imbalance.

▼ **Over-sampling (Upsampling): Add more examples to the minority class**

▼ **Random over-sampling**

Randomly make copies of the minority class until a ratio is reached.

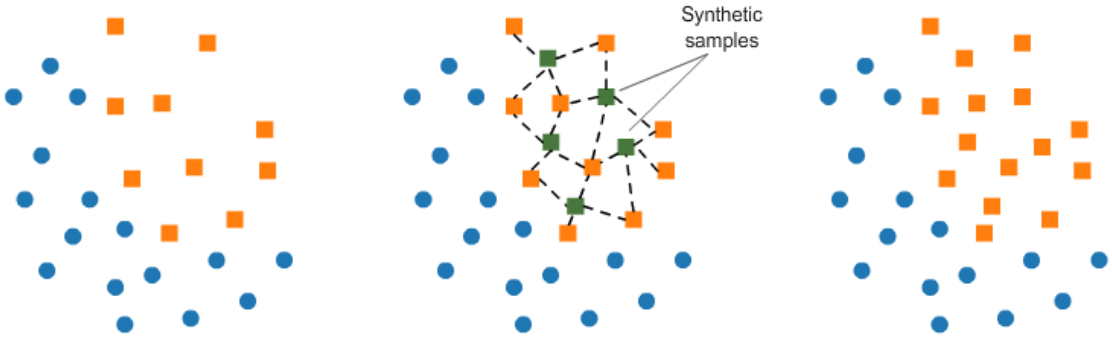


Credit: Rafael Alencar, https://www.kaggle.com/code/rafjaa/resampling-strategies-for-imbalanced-datasets/notebook

📌 Simply making replicas may make the model **overfit** to the few examples.

▼ **Generate synthetic examples**

SMOTE (synthetic minority oversampling technique) - creates synthetic examples of the rare class by combining original examples. It does this using a nearest neighbors approach.



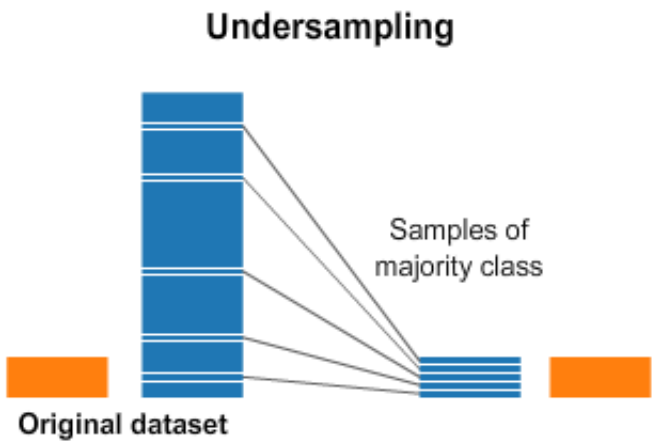
Credit: Rafael Alencar, https://www.kaggle.com/code/rafjaa/resampling-strategies-for-imbalanced-datasets/notebook

It can prevent the overfitting caused by random oversampling because it does not use original examples.

▼ **Under-samping (Downsampling): Remove examples from the majority class**

▼ **Random under-sampling**

Randomly remove samples of the majority class until a ratio is reached.

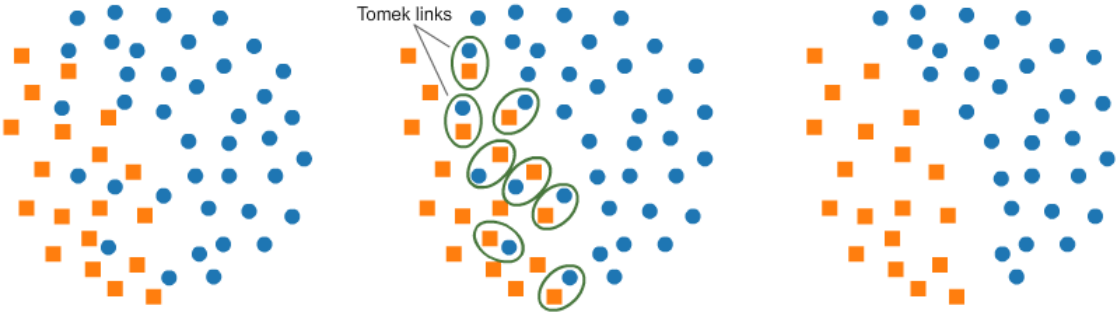


Credit: Rafael Alencar, https://www.kaggle.com/code/rafjaa/resampling-strategies-for-imbalanced-datasets/notebook

📍 Random under-sampling may make the resulting dataset too small for a model to learn from, so it only works when we have enough number of examples (at least thousands of samples) in the minority class.

▼ **Tomek links**

Find pairs of examples from opposite class that are close in proximity and remove the sample of the majority class in each pair.



Credit: Rafael Alencar, https://www.kaggle.com/code/rafjaa/resampling-strategies-for-imbalanced-datasets/notebook

It may help make the decision boundary more clear and the model learn the boundary better. But the model may not learn from the subtleties of the true decision boundary.

📍 Resampling method is a good starting point, but it runs the risk of overfitting training data (over-sampling) and losing important information from removing data (under-sampling).

▼ Model-level methods

- Make the model more robust to class imbalance.
- Does not change the distribution of the training data.

▼ **Update loss function**

Design a **loss function** that penalizes the wrong classifications of the minority class more than the wrong classifications of the majority class.

Force the model to treat specific classes with more weight than others during training.

- e.g. Class-balanced loss - make the weight of each class inversely proportional to the number of samples in that class.

  $W_i = \frac{n}{n_i}$

Class	Number of examples	Weight
A	1,000	1.01
B	10	101

  The loss caused by example x of class i: $L(x;\theta) = W_i \sum_j P(j|x;\theta) Loss(x,j)$

  where $Loss(x,j)$ is the loss when x is misclassified as class j (the wrong class).

▼ **Select appropriate algorithms**

- Tree-based models work well on tasks involving small and imbalanced datasets.
- Logistic regression is able to handle class imbalanced relatively well in a standalone manner. Adjust the probability threshold to improve the accuracy for predicting the minority class.

▼ **Combine multiple techniques**

1. **Under-sampling + ensemble**

   Use all samples of the minority class and a subset of the majority class to train multiple models and then ensemble those models.

Class	Number of examples	
A	1,000	divide into 10 groups with 100 examples each
B	100	Use all examples

2. **Under-sampling + update loss function**

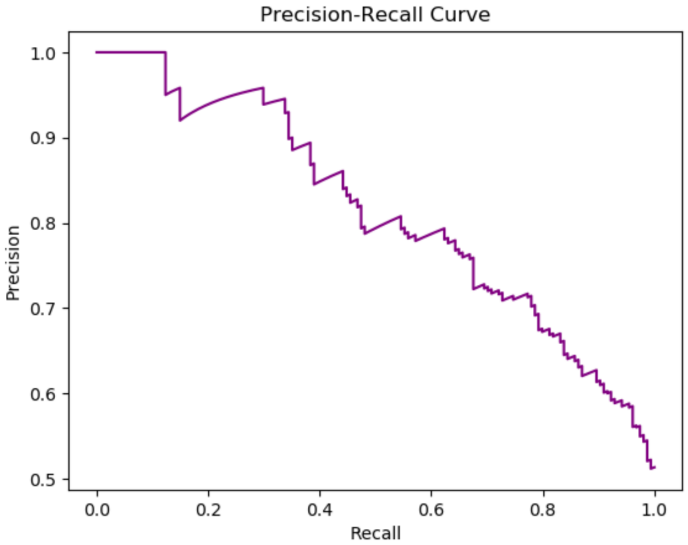
   Under-sample the majority class until a ratio is reached, calculate the new weights for both classes, then pass the new weights to the loss function of the model.

▼ Evaluation Metrics

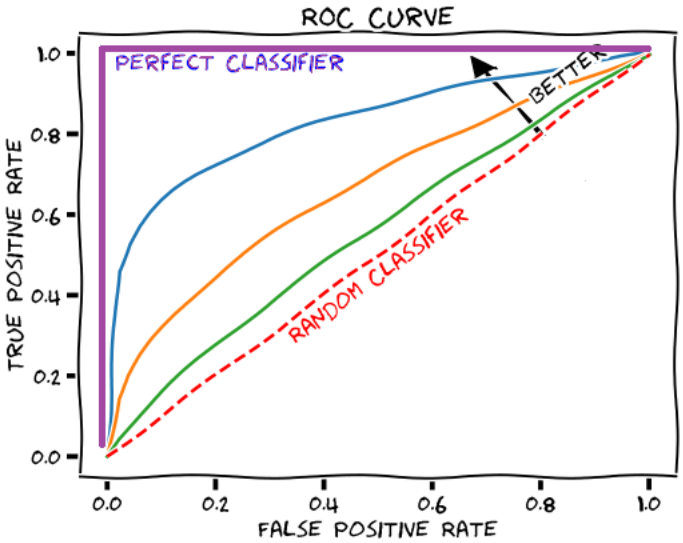
Choose appropriate evaluation metrics for the task.

📌 We should use **un-sampled data** instead of resampled data to evaluate the model because using the later will cause the model to overfit the resampled distribution.
The test data should provide an accurate representation of the original dataset.

- Accuracy is misleading when classes are imbalanced - performance of the model on the majority class will dominate the metric.
    - Consider using accuracy for each class individually.
- **Precision**, **recall**, and **F1** measure a model's performance with respect to the positive class in a binary classification problem.
- **Precision-Recall curve** - identify a threshold that works best for the dataset. It gives more importance to the positive class (put emphasis on how many predictions the model got right out of the total number it predicted to be positive), which is helpful for dealing with imbalanced data.



- **AUC of the ROC curve** - tune thresholds to increase recall and decrease false positive rate. It treats both classes equally and is less sensitive to model improvement on minority class, so it's less helpful compared to Precision-Recall curve.



Credit: Rachel Draelos, Source: https://glassboxmedicine.com/2019/02/23/measuring-performance-auc-auroc/